

Objektorientierte Programmierung (OOP)

Programmierparadigmen (eine Auswahl)



- **Strukturiere (prozedurale) Programmierung** (Dijkstra 1968)
 - if/then/else, do/while/until, function (statt goto statements)
 - Beispiele: Basic, C, Pascal, Fortran
- **Funktionale (deklarative) Programmierung** (Church 1936)
 - Variablen können nicht geändert werden (alles ist konstant)
 - Beispiele: Lisp, Haskell, Erlang, Clojure
- **Objektorientierte Programmierung** (Dahl und Nygaard 1966)
 - Klassen, Methoden, Objekte
 - Beispiele: Java, Scala, C#, C++, Ruby, Smalltalk

Viele Programmiersprachen erlauben inzwischen das Vermischen von Paradigmen.
Beispiele: Java, Scala, Javascript, C#, Python



Objektorientierte Programmierung

- Ursprung: [SIMULA](#) (1966) – Dahl und Nygaard + Hoare.
- Größter Influencer: [Smalltalk](#) (1972) – Alan Kay.
- Es wurden Objekte, Klassen, Unterklassen und virtuelle Methoden eingeführt. Klassen repräsentieren Domänenkonzepte und Unterklassen spezialisierte Konzepte.
- Ein Nebeneffekt war, dass Unterklassen den Code der Oberklasse erben bzw. wiederverwenden können.
- OOP soll beim Strukturieren und Design von Softwaresystemen helfen.

Siehe auch: „Object-oriented programming: some history, and challenges for the next fifty years”

<https://arxiv.org/abs/1303.0427>

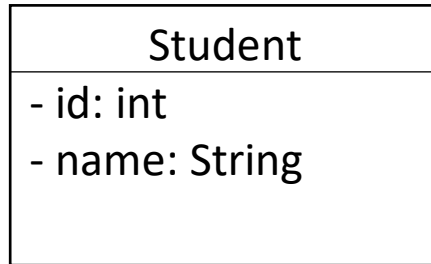
OOP – Konzepte

1. **Klassen** sind die Bausteine der OOP und definieren, welche Attribute und Methoden ihre Objekte erhalten. Eine Klasse dient als Blaupause für Objekte.
2. **Objekte** sind Instanzen von Klassen und haben einen Zustand und Methoden.
3. **Kapselung** bezieht sich auf das Konzept, dass Objekte ihren Zustand und Methoden vor „unbefugtem“ Zugriff schützen (z. B. private).
4. **Vererbung** ermöglicht es, dass eine Klasse die Attribute und Methoden einer anderen Klasse erbt (u. a. zur Code-Wiederverwendung).
5. **Polymorphismus** ermöglicht es, dass Methodenaufrufe an das korrekte Objekt gerichtet werden, auch wenn der konkrete Typ (aka Klasse) des Objekts vorm Aufruf nicht bekannt ist.
6. **Abstraktion** bezieht sich auf die Fähigkeit, komplexe Systeme zu vereinfachen, indem nur die wichtigen Details dargestellt werden. Dies wird oft durch die Verwendung von Interfaces oder abstrakten Klassen erreicht.

Klassen vs Objekte

Klassenwelt

```
public class Student {  
    private int id;  
    private String name;  
  
    public Student(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
}
```



Es gibt genau *eine* Klasse Student.

Objektwelt

Student stephi = new Student(1, "Stephi");

Object of Student

id: 1
name: Stephi

Student andre = new Student(2, "André");

Object of Student

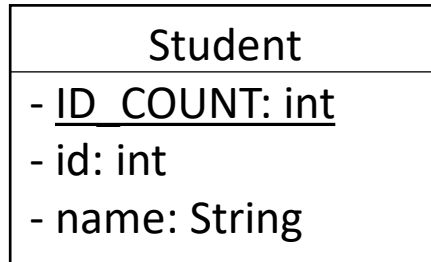
id: 2
name: André

Es gibt *beliebig viele* Objekte von der Klasse Student.
Jedes Objekt hat seinen *eigenen* Zustand (Instanz).

Klassen vs Objekte

Klassenwelt

```
public class Student {  
    private static int ID_COUNT = 0;  
    private int id;  
    private String name;  
  
    public Student(String name) {  
        this.id = ID_COUNT++;  
        this.name = name;  
    }  
}
```



Es gibt genau *eine* Klasse Student.
Jede Klasse hat ihren eigenen Zustand (static).

Objektwelt

Student stephi = new Student("Stephi");

Object of Student

ID_COUNT: 2
id: 1
name: Stephi

Student andre = new Student("André");

Object of Student

ID_COUNT: 2
id: 2
name: André

Es gibt *beliebig viele* Objekte von der Klasse Student.
Jedes Objekt hat seinen *eigenen* Zustand (non-static).

Klassen vs Objekte

Klassenwelt

Variable wird
einmalig in der
Klasse gespeichert.

```
public class Student {  
    → private static int ID_COUNT = 0;  
    private int id;  
    private String name;  
  
    public Student(String name) {  
        this.id = ID_COUNT++;  
        this.name = name;  
    }  
}
```

Student
- <u>ID_COUNT</u> : int
- id: int
- name: String

ID_COUNT ist **static** und gehört somit zur Klasse.
Es gibt somit genau *eine Variable ID_COUNT*.

Objektwelt

Student stephi = new Student("Stephi");

Object of Student

ID_COUNT: 2 ←
id: 1
name: Stephi

Student andre = new Student("André");

Object of Student

ID_COUNT: 2 ←
id: 2
name: André

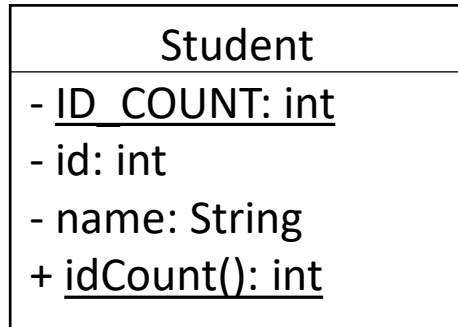
Alle Objekte teilen sich die
statischen Klassenvariablen.

Es gibt *beliebig viele* Objekte von der Klasse Student.
Jedes Objekt hat seinen *eigenen* Zustand (non-static).

Klassen vs Objekte

Klassenwelt

```
public class Student {  
    private static int ID_COUNT = 0;  
    private int id;  
    private String name;  
    // Constructor...  
    public static int idCount() {  
        return ID_COUNT;  
    }  
}
```



Objektwelt

`Student.idCount();` Greife auf statische Elemente über die Klasse zu.
(kann innerhalb der Klasse weggelassen werden).

`var andre = new Student("André");`
`andre.idCount();` Zugriff auf statische Elemente über Objekt möglich,
aber verwirrend: Klassenmethode oder Objektmethode?

Object of Student

ID_COUNT: 2
id: 1
name: Stephi

Object of Student

ID_COUNT: 2
id: 2
name: André

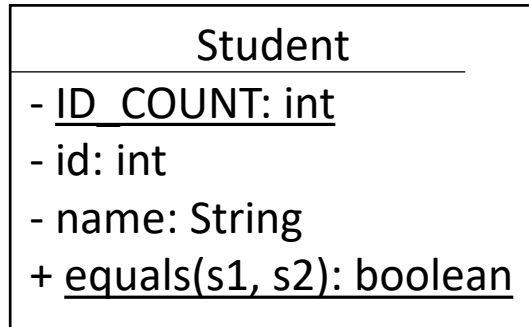
Statische Methoden haben *nur* Zugriff auf
statische Variablen und statische Methoden!
→ Die Methode `idCount()` hat nur Zugriff auf *ID_COUNT*.

Objekte haben auch Zugriff auf statische
Variablen und Methoden *ihrer* Klasse.

Klassen vs Objekte

Klassenwelt

```
public class Student {  
    private static int ID_COUNT = 1;  
    private int id;  
    private String name;  
    // Constructor...  
    public static boolean equals(Student s1,  
                                Student s2) {  
  
        return s1.id == s2.id;  
    }  
}
```



Objekte werden als
Parameter übergeben,
damit die statische
Methode auf die
Objektzustände
zugreifen kann.



new

new

Objektwelt

```
var stephi = new Student("Stephi");  
var andre = new Student("André");  
boolean sameStudents = Student.equals(stephi, andre);
```

Object of Student

ID_COUNT: 2
id: 1
name: Stephi

Object of Student

ID_COUNT: 2
id: 2
name: André

Statische Methoden können **nicht** auf **this** zugreifen.
Benötigte Objekte müssen als Parameter übergeben werden.

Objekte haben auch Zugriff auf statische
Variablen und Methoden *ihrer* Klasse.

Static

FYI: Wenn alles static wäre, wäre Java eine prozedurale Sprache.



Quelle: <https://eu.azcentral.com/story/entertainment/life/2019/11/13/what-causes-static-electricity-how-do-i-get-rid-of-it/4165052002/>

1. **Konstanten:** Konstanten werden oft als statische Variablen in einer Klasse definiert, da sie von allen *Objekten* dieser Klasse gemeinsam genutzt werden können.

Beispiel: die Konstanten PI und E in der Klasse Math.

2. **Utility-Klassen:** Utility- oder Hilfsklassen bieten allgemein nützliche Funktionen und sind oft zustandslos. Sie bestehen i. d. R. nur aus statischen Elementen z. B. Assert oder Math.

3. **Kein Zustand:** Wenn eine Methode auf keinen Zustand bzw. kein Objekt zugreift.

Beispiel: `List.of(...)` oder `public static void printHello() { System.out.println("Hello"); }`.

4. **Gemeinsame Eigenschaften oder Methoden:** Wenn mehrere Objekte einer Klasse dieselben Eigenschaften oder Methoden gemeinsam nutzen sollen, statt sie für jedes Objekt zu duplizieren.

Achtung Seiteneffekte: statische Variablen sind quasi globale Variable innerhalb der Klasse!

5. **Main-Methode:** Die Main-Methode einer Java-Anwendung muss statisch sein.

Merke: Statische Methoden können nur auf statische Variablen und Methoden zugreifen, aber nicht auf Objektvariablen und -methoden!

Kapselung

```
public class Cookbook {  
    private String name;  
    private List<Recipe> recipes = new ArrayList<>();
```

```
    public String getName() { return name; }
```

```
public List<Recipe> getRecipes() {  
    return recipes;  
}
```

```
public void setRecipes(List<Recipe> recipes) {  
    this.recipes = recipes;  
}
```

```
public void add(Recipe recipe) {  
    recipes.add(recipe);  
}
```

```
}
```

→

```
public Iterable<Recipe> getRecipes() {  
    return Collections.unmodifiableList(recipes);  
}
```

Klassen, Methoden und Variablen werden über **Zugriffsmodifikatoren** (**private**, **protected**, package-protected) gekapselt.

Verwandte Alternativen sind **Interfaces**, Objektkopien oder **unmodifiable Objects**.

Kapselung ist in den meisten OO-Sprachen optional.

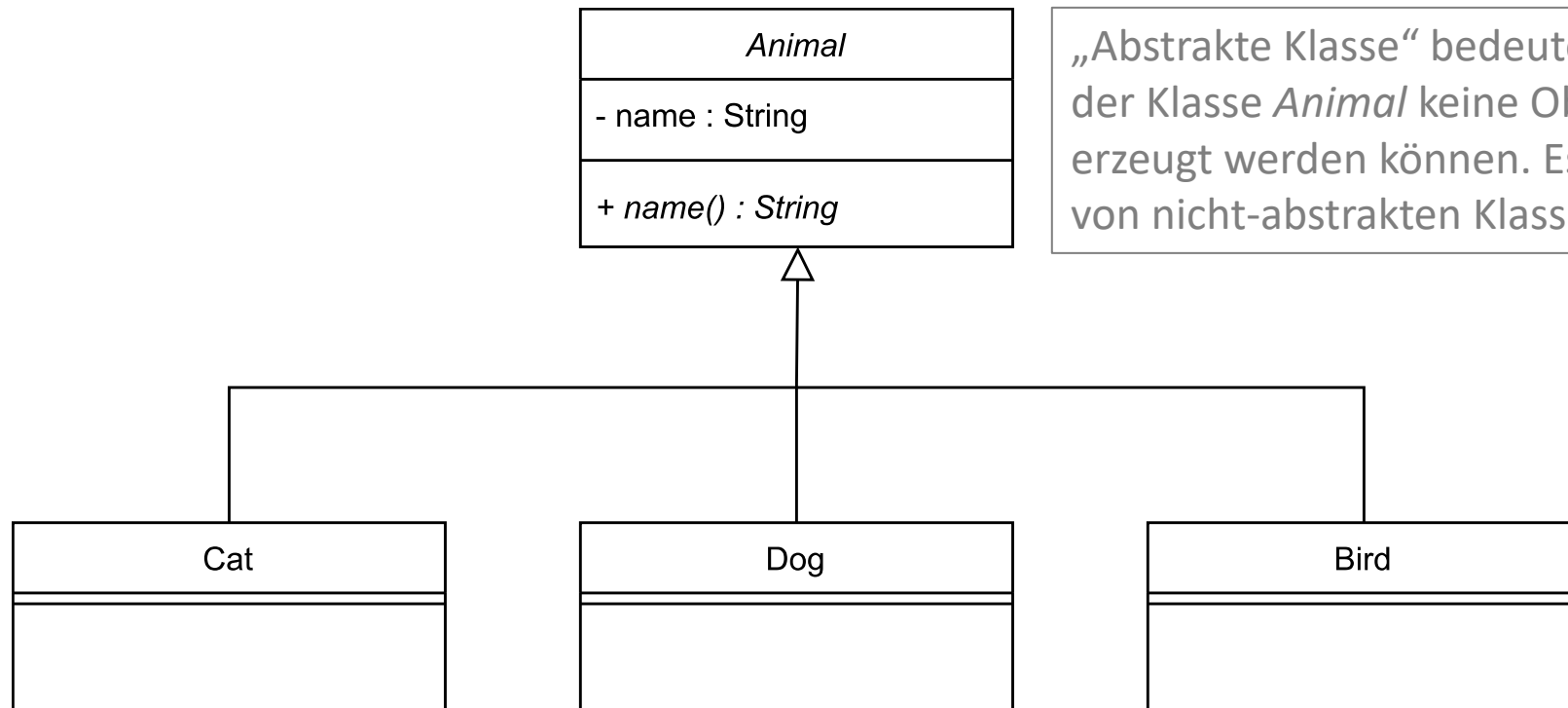
Es existiert wenig oder gar keine Pflicht zur Kapselung. Kapselung ist Aufgabe der Entwicklerin.

Siehe: Robert C. Martin (Clean Architecture, Seite 37)

Vererbung („Ist-Ein“)

„Ist-Ein“-Beziehung

Cat, Dog und Bird erben die Attribute und Methoden von der abstrakten Klasse Animal. In den Unterklassen kann man dann auf die nicht-privaten Attribute und Methoden von der Oberklasse Animal zugreifen.



„Abstrakte Klasse“ bedeutet lediglich, dass von der Klasse *Animal* keine Objekte mit `new` erzeugt werden können. Es ist auch möglich, von nicht-abstrakten Klassen zu erben.

Vererbung

„Ist-Ein“-Beziehung

Cat, Dog und Bird erben von Animal. Das heißt, ihre Objekte können auch das, was in der Klasse Animal definiert ist.

```
Cat cat = new Cat("Spot");  
System.out.print(cat.name());
```

1. Suche die Methode name()
in der Klasse des Objekts.

```
Dog dog = new Dog("Spike");  
System.out.print(dog.name());
```

```
Bird bird = new Bird("Spirit");  
System.out.print(bird.name());
```

Ausgabe: SpotSpikeSpirit

```
public abstract class Animal {  
    private String name;  
    public Animal(String name) {  
        this.name = name;  
    }  
    public String name() { return name; }  
}  
2. Suche weiter in der Oberklasse.  
public class Cat extends Animal {  
    public Cat(String name) { super(name); }  
}  
  
public class Dog extends Animal {  
    public Dog(String name) { super(name); }  
}  
  
public class Bird extends Animal {  
    public Bird(String name) { super(name); }  
}
```

Vererbung vs Interface

„Ist-Ein“-Beziehung

Eine Alternative zu abstrakten Klassen sind Interfaces.

In Interfaces sind nur Methoden und static final Variablen (Konstanten) erlaubt, aber keine Attribute.

BTW: Eine Klasse kann beliebig viele Interfaces implementieren.

```
Cat cat = new Cat("Spot");  
System.out.print(cat.name());
```

Die Methode name() wird direkt in
der Klasse des Objekts gefunden.

```
Dog dog = new Dog("Spike");  
System.out.print(dog.name());
```

```
Bird bird = new Bird("Spirit");  
System.out.print(bird.name());
```

Ausgabe: SpotSpikeSpirit

```
public interface Animal {  
    public String name();  
}
```

```
public class Cat implements Animal {  
    private String name;  
    public Cat(String name) { this.name = name; }  
    public String name() { return name; }  
}
```

```
public class Dog implements Animal {  
    private String name;  
    public Dog(String name) { this.name = name; }  
    public String name() { return name; }  
}
```

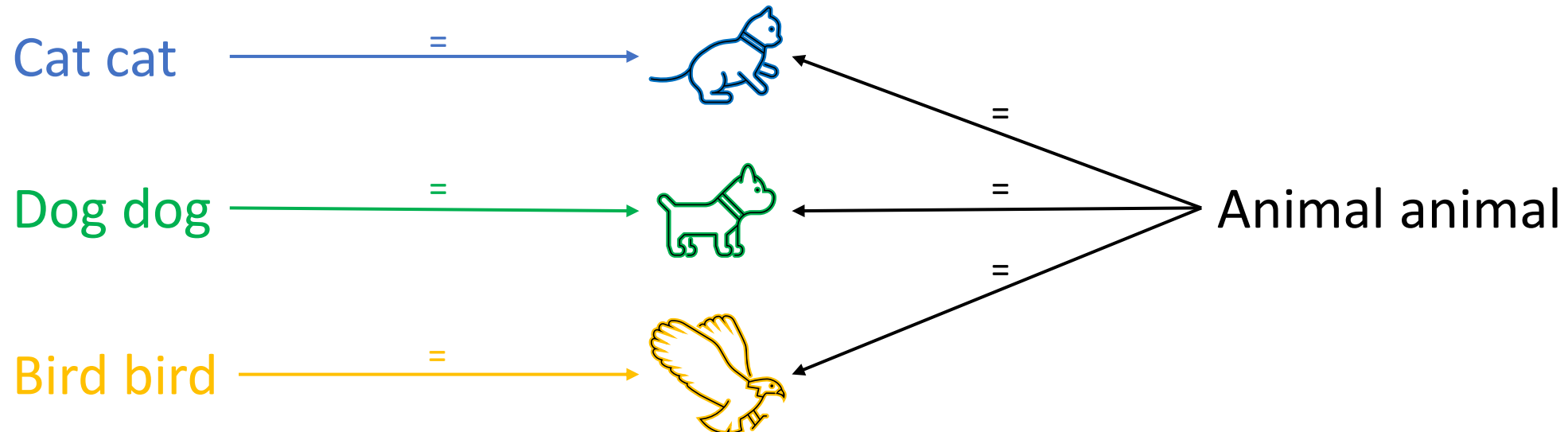
```
public class Bird implements Animal {  
    private String name;  
    public Bird(String name) { this.name = name; }  
    public String name() { return name; }  
}
```

Polymorphismus

„Ist-Ein“-Beziehung

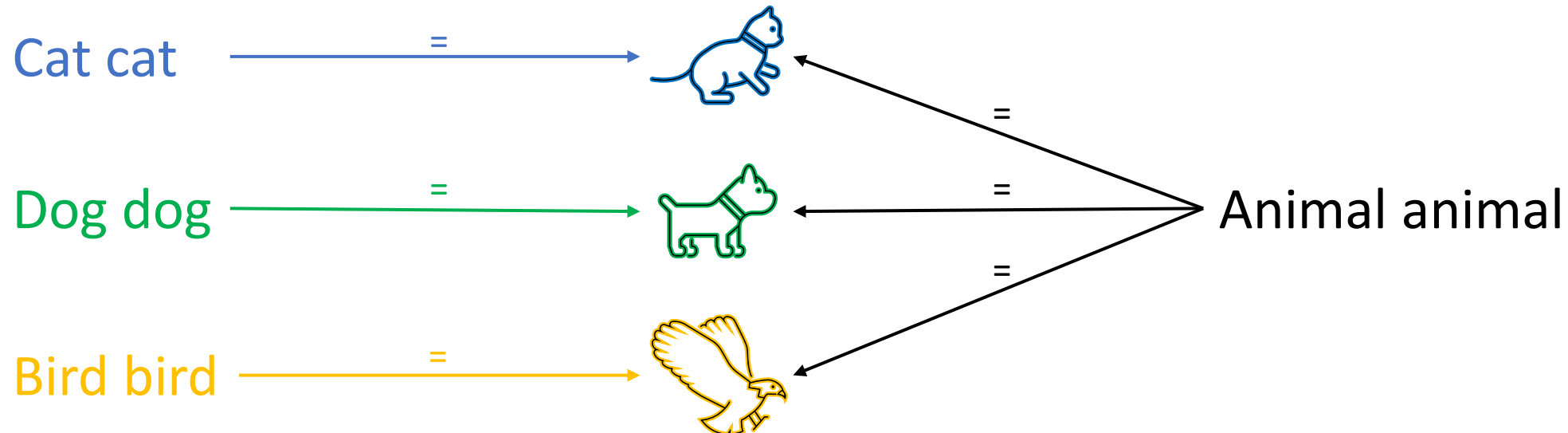
Durch die Vererbung sind Cat-, Dog- und Bird-Objekte auch ein Animal („Ist-Ein“).

Deshalb kann ich einer Animal-Variablen ein Cat-, Dog-, oder Bird-Objekt zuweisen.



Polymorphismus

- Durch den Variablentyp lege ich fest, wie ich auf ein Objekt schauen möchte (abstrakt oder konkret) und was ich damit alles machen kann.
- Eine *Cat-Variable* kann z. B. alles, was in der Oberklasse Animal definiert ist **und** alles, was in der Klasse Cat definiert ist!
- Eine *Animal-Variable* kann hingegen nur das, was in der Klasse Animal definiert ist, auch wenn das zugewiesene Cat-Objekt mehr könnte (Abstraktion).



Polymorphismus

„Ist-Ein“-Beziehung

Sobald ich für eine Variable die Oberklasse verwende, abstrahiere ich. Mir ist die konkrete Unterklasse dann „egal“.

```
Animal cat = new Cat("Spot");  
System.out.print(cat.name());
```

```
Animal dog = new Dog("Spike");  
System.out.print(dog.name());
```

```
Animal bird = new Bird("Spirit");  
System.out.print(bird.name());
```

Ausgabe: SpotSpikeSpirit

```
public abstract class Animal {  
    private String name;  
    public Animal(String name) {  
        this.name = name;  
    }  
    public String name() { return name; }  
}  
  
public class Cat extends Animal {  
    public Cat(String name) { super(name); }  
}  
  
public class Dog extends Animal {  
    public Dog(String name) { super(name); }  
}  
  
public class Bird extends Animal {  
    public Bird(String name) { super(name); }  
}
```

Polymorphismus

„Ist-Ein“-Beziehung

Sobald ich für die Variable `animal` die Oberklasse `Animal` verwende, kann ich nur `Animal`-Methoden aufrufen! Die Methode `meow()` aus der Klasse `Cat` ist für die Variable `animal` dann nicht verfügbar.

```
List<Animal> animals = List.of(new Cat("Spot")
                               new Dog("Spike")
                               new Bird("Spirit"));

for (Animal animal : animals)
    System.out.print(animal.name());
```

Ausgabe: SpotSpikeSpirit

```
public abstract class Animal {
    private String name;
    public Animal(String name) {
        this.name = name;
    }
    public String name() { return name; }
}

public class Cat extends Animal {
    public Cat(String name) { super(name); }
    public void meow() { return "Miau"; }
}

public class Dog extends Animal {
    public Dog(String name) { super(name); }
}

public class Bird extends Animal {
    public Bird(String name) { super(name); }
}
```

Polymorphismus

Das cat-Objekt befindet sich in einer Variablen von Typ **Cat**. Sie hat Zugriff auf alles, was in **Cat** und **Animal** verfügbar ist.

Geerbte
Methode von
Animal.

```
Cat cat = new Cat("Spot");
```

```
public String name()  
public String meow()
```



Das cat-Objekt befindet sich in einer Variablen von Typ **Animal**. Sie hat daher nur Zugriff auf das, was in **Animal** verfügbar ist.

```
Animal cat = new Cat("Spot");
```

```
public String name()
```



Polymorphismus

„Ist-Ein“-Beziehung

Bird überschreibt hier die Methode `name()`. Java sucht die Methode im Bottom-Up-Verfahren: erst in der Klasse des Objekts, dann in den Oberklassen.

```
List<Animal> animals = List.of(new Cat("Spot")
                               new Dog("Spike")
                               new Bird("Spirit"));
for (Animal animal : animals)
    System.out.print(animal.name());
```

Ausgabe: SpotSpikeSPIRIT

```
public abstract class Animal {
    protected String name;
    public Animal(String name) {
        this.name = name;
    }
    public String name() { return name; }
}

public class Cat extends Animal {
    public Cat(String name) { super(name); }
}

public class Dog extends Animal {
    public Dog(String name) { super(name); }
}

public class Bird extends Animal {
    public Bird(String name) { super(name); }
    public String name() {
        return name.toUpperCase();
    }
}
```

Polymorphismus

```
List<Animal> animals = List.of(new Cat("Spot")
                                new Dog("Spike")
                                new Bird("Spirit"));
```

```
for (Animal animal : animals) {
    if (animal instanceof Cat) {
        Cat cat = (Cat)animal;
        cat.meow();
    } else if (animal instanceof Dog) {
        Dog dog = (Dog)animal;
        dog.bark();
    } else if (animal instanceof Bird) {
        Bird bird = (Bird)animal;
        bird.chirp();
    } else {
        System.err.println("Unexpected animal!");
    }
}
```

Das Casten von Objekten in Java ist häufig ein "bad smell".
Dieser Code ist fehleranfällig, da häufig vergessen wird, das
switch-Statement anzupassen, sobald eine neue
Unterklasse (z. B. Crocodile) hinzugefügt wird.



```
public abstract class Animal {
    ...
    public String name() { return name; }
}

public class Cat extends Animal {
    public void meow() {
        System.out.print("Miau");
    }
}

public class Dog extends Animal {
    public void bark() {
        System.out.print("Wuff");
    }
}

public class Bird extends Animal {
    public void chirp() {
        System.out.print("Piep");
    }
}
```

Ausgabe: MiauWuffPiep

Polymorphismus

If-Abfragen mit `instanceof` sind häufig ein Zeichen dafür, dass Unterklassen und Methoden nicht gut designed wurden (schlecht erweiterbarer Code). In diesem Beispiel können neue Unterklassen hinzugefügt werden (z. B. Crocodile), ohne die for-Schleife ändern zu müssen:

```
List<Animal> animals = new ArrayList<>();
animals.add(new Cat("Spot"));
animals.add(new Dog("Spike"));
animals.add(new Bird("Spirit"));
// animals.add(new Crocodile("Croc"));

for (Animal animal : animals) {
    animal.makeNoise(); 🐾
}
```

```
public abstract class Animal {
    ...
    public String name() { return name; }
    public abstract void makeNoise();
}

public class Cat extends Animal {
    public void makeNoise() {
        System.out.print("Miau");
    }
}

public class Dog extends Animal {
    public void makeNoise() {
        System.out.print("Wuff");
    }
}

public class Bird extends Animal {
    public void makeNoise() {
        System.out.print("Piep");
    }
}

Ausgabe: MiauWuffPiep
```

Function Overloading

Der „kleine Bruder“ des Polymorphismus

```
public record Cat(String name) {
```

```
    public void setOwner(Person person) {  
        this.owner = person;  
    }
```

```
    public void setOwner(PetShop petShop) {  
        this.owner = petShop;  
    }
```

```
    public void move(int distance, String direction) {  
        System.out.println(name + " has moved " + distance + " meters in the " + direction + " direction.");  
    }
```

```
    public void move(int distance) {  
        System.out.println(name + " has moved " + distance + " meters.");  
    }  
}
```

```
public static void main(String[] args) {  
    Cat cat = new Cat("Garfield");  
    Person person = new Person("Bob");  
    cat.setOwner(person);  
    cat.move(10, "North");  
}
```

Die Methode wird anhand der Parameter erkannt.

Typ stimmt überein.

Typen stimmen überein.

Gleicher Methodennamen, unterschiedliche Parameter.

OOP-Kritik

1. **Komplexität:** OOP kann komplex werden, insbesondere wenn es um komplexe Hierarchien von Klassen und Vererbung geht.
2. **Parallelisierung:** OOP kann sich schwieriger parallelisieren lassen, insbesondere wenn es um die Verwaltung gemeinsamer Zustände geht.
3. **Starre Struktur:** OOP-basierte Systeme können starr sein und es kann schwierig sein, Änderungen an der Struktur vorzunehmen, ohne den gesamten Code zu beeinträchtigen.
4. **Fehlersuche:** Da OOP-Systeme aus vielen kleinen Klassen bestehen, kann es schwierig sein, den Ursprung eines Fehlers zu identifizieren.
5. **Performance:** OOP erfordert zusätzlichen Code-Overhead, da Klassen und Objekte erstellt und verwaltet werden müssen. Dies kann zu höheren Speicher- und Leistungsanforderungen führen (häufig vernachlässigbar aufgrund moderner Compiler). Aber: Polymorphismus erschwert Compiler-Optimierungen.

There is No Silver Bullet

„Essentielle Komplexität wird durch das zu lösende Problem verursacht und kann durch nichts beseitigt werden; wenn Benutzer wollen, dass ein Programm 30 verschiedene Dinge tut, dann sind diese 30 Dinge wesentlich und das Programm muss diese 30 verschiedenen Dinge tun.“

Fred Brooks

<http://worrydream.com/refs/Brooks-NoSilverBullet.pdf>

Das bedeutet, dass ein komplexes Problem komplex bleibt, egal ob man es objekt-orientiert, daten-orientiert, funktional oder prozedural löst. Manche Probleme lassen sich eventuell mit einem bestimmten Paradigma eleganter lösen als andere Probleme.

Oder kurz: „[Wer als Werkzeug nur einen Hammer hat, sieht in jedem Problem einen Nagel](#)“